Отказ от gob в Data

Развитие CPU-гипотезы H-PERF2-6-1

Сохранённый ответ от 4 октября 2026 года. Статус: предложение.

Рекомендация: сохранить H-PERF2-6-1 как завершённый эксперимент и зарегистрировать новую гипотезу об отказе от gob в Data. Сейчас измерена замена RPC-конверта; полезная нагрузка, журнал и снимки всё ещё используют gob.

В исходном документе [1] есть основание для следующего эксперимента, но причинность некоторых выводов сформулирована слишком категорично. База анализа — stage6, коммит `9f42180`; полный SHA:

`9f4218056f663c1884de0e2294283319f786bf22`

1. Проверять CPU на полезную операцию

Доля RPC уменьшилась, хотя суммарное CPU трёх узлов по таблице выросло с **43,59 до 50,56 с — примерно на 16%**. Поэтому основной показатель предлагается определить так:

$$C_{\text{cluster}} = \frac{\sum_i T_{\text{CPU},i}}{N_{\text{success}}}.$$

Здесь $T_{\text{CPU},i}$ — CPU-время узла i за измерительное окно, а N_{success} — число успешных клиентских операций за то же окно. Единица измерения: CPU-секунды на операцию.

Дополнительно измерять CPU обработки Data на одну команду, аллокации, сетевые байты и задержки. Долю RPC оставить диагностическим показателем.

37% RPC-пути — это около 10,2% всего CPU лидера. Вместе с предварительным кодированием наблюдаемые затраты составляют около 14,2%: $(2{,}47 + 0{,}95)/24{,}15$. Эти числа показывают область оптимизации, но ещё не обещают соответствующего ускорения.

2. Убрать повторное кодирование вместе с gob

В текущем коде:

- `MeasureData` полностью кодирует значение и оставляет только длину [2].
- `appendEntry` снова кодирует Data при построении запроса [3].
- `encodeLogData` отдельно кодирует её для хранения [4].

Предлагаемое направление: **один раз получить неизменяемое представление команды, проверить его длину и переиспользовать при отправках соседям и повторах**. Для нового фиксированного формата точный размер также можно вычислять без сериализации.

Независимый gob-поток на каждую запись теряет преимущество передачи описаний типов в длительно живущем потоке. Этот механизм согласуется с документацией `encoding/gob` [5], но величину его вклада нужно подтвердить отдельным сравнением.

3. Задать явный формат и границы совместимости

Для KV подходит небольшой версионированный формат с видом операции, ID, длинами и содержимым `Key`, `Value`, `CompareValue`. Поля результата следует отделить от передаваемой команды после проверки существующего контракта.

При этом Raft принимает `any`, а FSM ожидает конкретный `Command`. Поэтому предлагается подключаемый кодек приложения: KV знает формат команды, Raft — версию и непрозрачные байты. Это **предложение архитектуры**; существующего такого интерфейса при анализе не обнаружено.

Просто передавать `[]byte` недостаточно: текущие транспорт и хранилище снова обернут их в gob. Нужны явное представление байтов без gob, правила неизвестных версий и сохранение различий между `nil`, пустыми данными, командой и конфигурацией.

4. Разделить эксперимент на варианты

Вариант	Что меняется	Что проверяем
A	Текущая stage6 `9f42180`	База
B	Явный кодек Data, прежняя частота кодирования	Эффект замены gob
C	Кодек B плюс переиспользование закодированных данных	Дополнительный эффект устранения повторов
D	Новый формат хранения	Дополнительный эффект отказа от gob в журнале

Журнал и снимки требуют собственной версии формата, проверки восстановления старых данных и плана миграции. Их лучше вводить отдельным шагом, чтобы видеть источник результата.

5. Предлагаемая формулировка гипотезы

> **H-PERF3-DATA-1, предложение.** Относительно сборки stage6 `9f42180` замена gob-кодирования Data на явный версионированный формат снижает CPU сериализации и десериализации одной команды минимум на 25%. Переиспользование закодированного представления проверяется отдельным вариантом. Практический эффект подтверждается снижением суммарного CPU кластера на успешную операцию при одинаковой нагрузке, сохранении корректности и выполнении заранее заданных ограничений задержек.

25% здесь — предлагаемый целевой порог, а не прогноз. Минимально полезное снижение общего CPU тоже нужно зафиксировать до измерений.

Для новой серии заранее определить число повторов, чередование A/B/C, одинаковое исходное состояние, профиль нагрузки и правила обработки всех попыток. Старый L3 оставить историческим сравнением; основная контрольная сборка — текущая stage6. Один профиль на вариант не показывает межпрогонную вариативность.

В исходном документе утверждение «gob вызвал весь рост CPU ведомых» предлагается заменить на: **«Наблюдения согласуются с удорожанием обработки Data; причинный вклад требует отдельного эксперимента»**. Результат H-PERF2-6-1 при этом сохраняется без пересмотра порогов.

Источники и границы вывода

Числа в этом документе взяты из сохранённого заключения H-PERF2-6-1 и его сравнительных таблиц. Производные проценты рассчитаны по этим числам. Новых измерений, изменений реализации и повторного профилирования при подготовке ответа не выполнялось.

Факты: текущий путь кодирования Data, повторная сериализация для измерения длины, приведённые значения таблиц CPU.

Интерпретация: независимые gob-потоки и повторное кодирование являются обоснованными кандидатами на оптимизацию; их причинный вклад в общий рост CPU ещё требует отдельной проверки.

Предложение: новый кодек, переиспользование представления, варианты A–D и целевой порог 25%. Это не принятое архитектурное решение и не разрешение на изменение форматов или запуск стенда.

Все локальные пути ниже указаны относительно корня репозитория `raft`.

[1] `docs/TODO.CPU-hypothesis-H-PERF2-6-1.md`.
Исходная гипотеза, вердикт и сравнительные таблицы; раздел «Интерпретация» начинается со строки 88.

[2] `pkg/raft/protocol/data.go`, строки 30 и 50.
`EncodeData` создаёт независимый gob-поток; `MeasureData` вызывает кодирование и возвращает длину.

[3] `pkg/raft/protocol/request.go`, строка 141.
`appendEntry`: кодирование Data при построении AppendEntries.

[4] `pkg/raft/store/log_format.go`, строка 273.
`encodeLogData`: отдельное gob-кодирование Data для журнала.

[5] Go, документация пакета `encoding/gob`.
`https://pkg.go.dev/encoding/gob`
Раздел Overview: поток значений и повторное использование Encoder. Обращение: 4 октября 2026 года.

[6] `pkg/raft/contract/rpc.go`, строка 141;
`raft_cm_leader.go`, строка 63;
`pkg/kvservice/kvservice.go`, строка 249.
Контракты `LogEntry.Data`, `Apply` и применения `Command` к FSM.